

7.1 Anatomía de una Función en Dart y Firma Tipada

Una función en Dart se compone de una firma con tipo de retorno explícito, un nombre identificador, una lista de parámetros delimitada entre paréntesis y un cuerpo delimitado por llaves `{}`.

```
double calcularPromedio(double nota1, double nota2) {  
  return (nota1 + nota2) / 2;  
}
```

Si una función no retorna ningún valor explícito, se especifica el tipo de retorno como `void`.

7.2 Parámetros Posicionales vs. Parámetros Nombrados y Opcionales

Dart ofrece una flexibilidad excepcional para estructurar los argumentos de entrada de una función, dividiéndolos en tres categorías principales:

1. Parámetros Posicionales Obligatorios

Son los parámetros estándar. El orden en el que se pasan los argumentos en la llamada debe coincidir exactamente con el orden definido en la firma.

```
void registrarAlumno(String boleta, String nombre) { ... }  
// Llamada: registrarAlumno('2021630000', 'Carlos');
```

2. Parámetros Nombrados (`{ }`)

Se delimitan con llaves `{}`. Mejoran drásticamente la legibilidad del código al hacer explícito el nombre de cada argumento en la llamada. Por defecto son opcionales y nulos, pero pueden marcarse como **obligatorios** usando la palabra clave `required` o asignarles un **valor por defecto**.

```
void configurarServidor(  
  required String host,  
  int puerto = 8080, // Valor por defecto si no se pasa  
  String? token,    // Anulable opcional  
) { ... }  
  
// Llamada:  
configurarServidor(host: '127.0.0.1', token: 'XYZ123');
```

3. Parámetros Posicionales Opcionales (`[ ]`)

Se delimitan con corchetes `[]` al final de la lista de parámetros. Permiten definir argumentos posicionales que pueden omitirse o tomar un valor por defecto.

```
void generarReporte(String titulo, [String formato = 'PDF', bool imprimir = false]) { ... }

// Llamadas válidas:
generarReporte('Ficha Académica');
generarReporte('Ficha Académica', 'CSV', true);
```

7.3 Funciones de Flecha (*Arrow Functions*) y Funciones Anónimas

Sintaxis de Flecha (`=>`)

Cuando una función consta de **una sola expresión** para evaluar y retornar, podemos sustituir el bloque `{ return ...; }` por el operador de flecha `=>`:

```
// Función estándar:
bool esAprobatorio(double nota) {
  return nota >= 6.0;
}

// Equivale exactamente con flecha:
bool esAprobatorio(double nota) => nota >= 6.0;
```

Funciones Anónimas (Lambdas)

Son funciones sin un nombre explícito que se definen al vuelo, utilizadas frecuentemente como callbacks en métodos de colecciones o en manejadores de eventos en Flutter (`onPressed`):

```
final numeros = [1, 2, 3];
numeros.forEach((n) => print('Número: $n'));
```

7.4 Scope, Closures y Funciones de Orden Superior

Lexical Scope (Alcance Léxico)

Dart determina el alcance de las variables basándose en la estructura física del código. Una variable es accesible únicamente dentro del bloque delimitado por llaves `{ }` donde fue declarada (y sus bloques anidados).

Closures (Clausuras)

Un *closure* es una función objeto que tiene acceso a las variables definidas en su alcance léxico externo, incluso cuando la función se ejecuta **fuera** de ese alcance original.

```
/// Función que retorna otra función (Closure)
Function crearContador() {
  int contador = 0; // Variable capturada por el closure
  return () {
    contador++;
    return contador;
  };
}

void main() {
  var miContador = crearContador();
  print(miContador()); // 1
  print(miContador()); // 2
}
```

Funciones de Orden Superior (*Higher-Order Functions*)

Son funciones que toman a otra función como parámetro o devuelven una función como resultado.

7.5 Ejemplo

Creen el archivo `bin/d7_funciones.dart` en VS Code. El siguiente programa demuestra la declaración de funciones con parámetros nombrados, opcionales, closures y funciones de orden superior para procesar datos de la ESCOM.

```
/// Módulo D7: Funciones y Closures en Dart
/// Demostración práctica de firmas tipadas, parámetros nombrados, closures y
/// Higher-Order Functions.

void main() {
  print('=====');
  print('      MOTOR DE CÁLCULO Y FUNCIONES AVANZADAS - ESCOM      ');
  print('=====\\n');

  // 1. Llamada a función con parámetros nombrados y valores por defecto
  print('--- 1. Configuración de Parámetros Nombrados ---');
  final String statusApi = generarUrlApi(
    endpoint: '/estudiantes/evaluar',
    puerto: 443,
    usarHttps: true,
  );
  print('URL generada: $statusApi\\n');
```

```
// 2. Uso de Funciones de Orden Superior pasando una callback
print('--- 2. Procesamiento de Lista con Callback de Orden Superior ---');
final List<double> notasRaw = [5.5, 8.0, 9.5, 4.0, 7.0];

// Invocamos la función pasando una función anónima como filtro
final List<double> notasAprobadas = procesarNotas(
    notas: notasRaw,
    criterioFiltro: (nota) => nota >= 6.0,
);

print('Notas originales: $notasRaw');
print('Notas aprobadas filtradas: $notasAprobadas\n');

// 3. Demostración de Closure (Captura de Estado Léxico)
print('--- 3. Generación de Función Calculadora de Ponderación (Closure) ---');

// Creamos dos funciones de ponderación específicas reteniendo el factor en
memoria
final calcularExamen70 = crearCalculadorPonderado(factor: 0.70);
final calcularTareas30 = crearCalculadorPonderado(factor: 0.30);

final double notaExamen = 9.0;
final double notaTareas = 10.0;

final double puntosExamen = calcularExamen70(notaExamen);
final double puntosTareas = calcularTareas30(notaTareas);
final double notaFinalCalculada = puntosExamen + puntosTareas;

print('Nota Examen (9.0 al 70%): $puntosExamen pts');
print('Nota Tareas (10.0 al 30%): $puntosTareas pts');
print('Nota Final Ponderada: $notaFinalCalculada pts');

print('\n=====');
}

/// Función que construye una URL con parámetros nombrados y opcionales.
String generarUrlApi({
    required String endpoint,
    String host = 'api.escom.ipn.mx',
    int puerto = 80,
    bool usarHttps = false,
}) {
    final String protocolo = usarHttps ? 'https' : 'http';
    return '$protocolo://$host:$puerto$endpoint';
}

/// Función de Orden Superior que recibe una función `bool Function(double)` como
callback.
List<double> procesarNotas({
    required List<double> notas,
    required bool Function(double) criterioFiltro,
}) {
    final List<double> resultado = [];
```

```
for (final nota in notas) {
  if (criterioFiltro(nota)) {
    resultado.add(nota);
  }
}
return resultado;
}

/// Función que retorna un Closure (función anónima) que retiene la variable
`factor`.
double Function(double) crearCalculadorPonderado({required double factor}) {
  return (double calificacion) => calificacion * factor;
}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d7_funciones.dart
```

7.6 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para dominar el diseño de funciones y closures.

Ejercicio 1: Calculadora de Impuestos y Becas con Parámetros Nombrados

Crea una función llamada `calcularMontoBeca` que determine el apoyo económico mensual de un estudiante en ESCOM:

1. Define los siguientes parámetros nombrados:
 - `double montoBase` (obligatorio con `required`).
 - `double promedio` (obligatorio con `required`).
 - `bool esForaneo` (opcional con valor por defecto `false`).
2. Regla de negocio:
 - Si el `promedio` es `>= 9.0`, incrementa la beca base en un `25%`.
 - Si `esForaneo` es `true`, agrega un bono fijo de `$1,000.0` MXN al total.
3. Imprime la llamada a la función para 3 perfiles distintos de alumnos.

Ejercicio 2: Generador de Acumuladores Financieros (Closure)

Escribe un script en Dart que simule el control de saldo de una caja de ahorro estudiantil mediante un *closure*:

1. Crea una función `crearCajaAhorro({double saldoInicial = 0.0})` que declare una variable interna `saldo`.
 2. La función debe retornar un closure de tipo `double Function(double deposito)` que sume el depósito al saldo actual y devuelva el saldo acumulado actualizado.
 3. En la función `main`, crea dos cajas de ahorro independientes (`cajaGrupoA` y `cajaGrupoB`), realiza múltiples depósitos en cada una y demuestra que cada instancia conserva su propio estado en memoria sin interferir entre sí.
-